Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence Practical

Practical – II

Roll No. : T045	Name: Omkar Thukarul
Class : TYIT	Date/Time of Submission : 17/07/2026

Aim: Problem Solving by Searching (Uninformed Search)

1). Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the shortest sequence of moves to reach the goal state using Breadth-First Search (BFS).

SOURCE:

```
from collections import deque
# Function to display puzzle state def print_state(state):
    for i in range(0, 9, 3):
      print(state[i], state[i + 1], state[i + 2])
print()
# Function to find possible next states def
get_neighbors(state):
   neighbors = []
   # Find position of blank tile
blank_index = state.index(0)
   # Possible moves
moves = {
      "Up": -3,
      "Down": 3,
      "Left": -1,
      "Right": 1
   }
   for move, position_change in moves.items():
new_index = blank_index + position_change
      # Check valid moves       if move == "Up" and
blank_index < 3:          continue       if move ==
"Down" and blank_index > 5:          continue       if
move == "Left" and blank_index % 3 == 0:
continue       if move == "Right" and blank_index % 3
== 2:          continue       # Create new state
new_state = list(state)       new_state[blank_index],
new_state[new_index] = \
new_state[new_index], new_state[blank_index]
neighbors.append((tuple(new_state), move))    return
neighbors # BFS Function def bfs(initial_state,
goal_state):    queue = deque()
```

OUTPUT:

```
Initial State:
1 2 3
4 0 6
7 5 8

Goal State:
1 2 3
4 5 6
7 8 0

Shortest sequence of moves:
Step 1 - Move: Down
1 2 3
4 5 6
7 0 8

Step 2 - Move: Right
1 2 3
4 5 6
7 8 0

Goal reached successfully.
Total number of moves: 2
Omkar 45
```

```
queue.append((initial_state, []))     visited = set()
visited.add(initial_state)     while queue:
       current_state, path = queue.popleft()
if current_state == goal_state:
           return path         for neighbor, move in
get_neighbors(current_state):             if neighbor not in visited:
visited.add(neighbor)                 queue.append((neighbor, path
+ [(move, neighbor)]))     return None initial_state = (      1, 2,
3,
   4, 0, 6,
   7, 5, 8
)
goal_state = (
   1, 2, 3,
   4, 5, 6,
   7, 8, 0 ) print("Initial
State:")
print_state(initial_state)

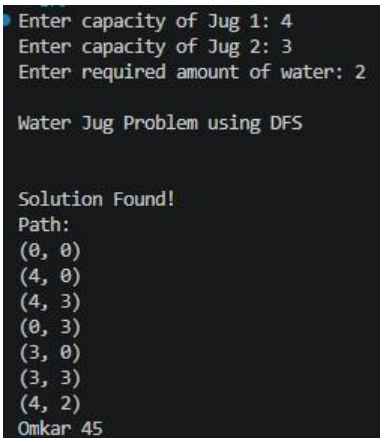
print("Goal State:")
print_state(goal_state) solution =
bfs(initial_state, goal_state) if
solution:
   print("Shortest sequence of moves:")
current_step = 1     for move, state in
solution:
       print("Step", current_step, "- Move:", move)
print_state(state)         current_step += 1
   print("Goal reached successfully.")
print("Total number of moves:", len(solution))
else:
   print("No solution found.")
print("Omkar 45")
```

2). Given two water jugs of 4 litres and 3 litres capacity, write a Python program to obtain exactly 2 litres in one jug using Depth-First Search (DFS). [Vary capacity of jugs]

SOURCE:

```
jug1 = int(input("Enter capacity of Jug 1: "))
jug2 = int(input("Enter capacity of Jug 2: "))
goal = int(input("Enter required amount of water: "))
visited = set() def dfs(state, path):     if state in visited:
return False     visited.add(state)     path.append(state)
x, y = state     if x == goal or y == goal:         print("\nSolution Found!")
       print("Path:")         for p in path:
print(p)         return True     next_states = []
next_states.append((jug1, y))
next_states.append((x, jug2))
next_states.append((0, y))
next_states.append((x, 0))     transfer = min(x,
```

OUTPUT:



```
jug2 - y)     next_states.append((x - transfer, y +
transfer))     transfer = min(y, jug1 - x)
next_states.append((x + transfer, y - transfer))
for new_state in next_states:         if
dfs(new_state, path.copy()):
          return True return False
print("\nWater Jug Problem using DFS\n")
if not dfs((0, 0), []):
   print("No Solution Exists")
```

3). Given a weighted graph representing cities and distances between them, write a Python program to find the least-cost path from Arad to Bucharest using Uniform Cost Search (UCS). [Provide any other weighted graph for applying UCS] **SOURCE:**

```
import heapq graph
= {
   'Mumbai': [('Pune', 150), ('Nashik', 170)],
   'Pune': [('Mumbai', 150), ('Nashik', 210), ('Aurangabad', 235)],
   'Nashik': [('Mumbai', 170), ('Pune', 210), ('Aurangabad', 180)],
   'Aurangabad': [('Pune', 235), ('Nashik', 180), ('Nagpur', 470)],
   'Nagpur': [('Aurangabad', 470)]
} def uniform_cost_search(graph, start,
goal):
   priority_queue = [(0, start, [start])]
visited = set()     while
priority_queue:
       cost, city, path = heapq.heappop(priority_queue)
if city in visited:             continue
visited.add(city)        if city == goal:             return
path, cost         for neighbor, distance in graph[city]:
if neighbor not in visited:
             heapq.heappush(priority_queue, (cost + distance, neighbor, path + [neighbor]))
return None, float('inf') start = "Mumbai" goal = "Nagpur" path, cost =
uniform_cost_search(graph, start, goal) print("Uniform Cost Search (India)")
print("Start City :", start) print("Goal
City  :", goal)
print("Least Cost Path :", " -> ".join(path))
print("Total Cost :", cost, "km")
print("Omkar 45")
```

OUTPUT:

```
Uniform Cost Search (India)
Start City : Mumbai
Goal City  : Nagpur
Least Cost Path : Mumbai -> Nashik -> Aurangabad -> Nagpur
Total Cost : 820 km
Omkar 45
```